COGANT: Codebase-to-GNN Translation

Theory of the Program Graph IR and Practice of the Python Pipeline

Daniel Ari Friedman

Active Inference Institute

daniel@activeinference.institute

ORCID: 0000-0001-6232-9096

April 9, 2026

Contents

1 Abstract

COGANT (Codebase-to-GNN Translation) is a system for turning software repositories into structured Active Inference artifacts expressed in the Active Inference Institute's **Generalized Notation Notation** (GNN)[1]. It occupies the space between classical program analysis—which already reasons over graphs such as call graphs and code property graphs [Yamaguchi et al., 2014]—and data-driven methods that expect tensors, consistent node and edge vocabularies, and exportable training bundles [Allamanis et al., 2018, Wu et al., 2021, Scarselli et al., 2009].

The design centers on a **program graph IR**: nodes denote entities (for example functions and variables), edges denote relations (for example calls and data flow), and each assertion carries **confidence** and **provenance** so that downstream models can treat uncertainty explicitly. A **fixpoint translation engine** applies declarative rules iteratively until no new semantic mappings emerge, resolves overlapping mappings by confidence, and reports coverage gaps explicitly. **Dynamic enrichment** from coverage reports and execution traces promotes mappings from a `STATIC_ONLY` tier to `STATIC_PLUS_RUNTIME` when runtime evidence corroborates static inference. A **Python orchestration layer** (session and pipeline APIs, CLI, configuration) performs discovery, parsing, normalization, graph construction, translation rules, state-space compilation, validation, and export. A **Rust core** (organized as separate crates in the package tree) holds graph operations, translation, state-space structures, Generalized Notation Notation formatting, and PyO3 bindings; the authoritative **implementation status** distinguishes what is wired in Python today from staged or planned native acceleration (see `../cogant/docs/SPEC.md`).

This manuscript, parallel to the developer documentation under `../cogant/docs/`, states the theory of that IR and the practice of running the pipeline, configuring Generalized Notation Notation exports (canonical GNN Markdown, GNN JSON, and interop targets such as GraphML, Parquet, and framework-specific tensor formats), and extending the tool through parsers, rules, validators, and exporters. It is written to remain valid whether this tree stays next to the package (`../cogant/`) or is later promoted under `../../../projects/` for full template pipeline rendering.

[1]Throughout this manuscript **GNN** refers to the Active Inference Institute's **Generalized Notation Notation** (repository), a structured notation for state-space and process models—*not* graph neural networks. Downstream consumers, including graph neural network training pipelines, can ingest the emitted artifacts, but COGANT itself produces Generalized Notation Notation bundles.

2 Introduction and Scope

Machine learning on source code has matured alongside static analysis: both need structured representations of programs, but learning pipelines additionally need fixed feature layouts, batchable graphs, and reproducible exports [Allamanis et al., 2018]. COGANT addresses that gap by making the path from repository checkout to **Generalized Notation Notation (GNN) bundles** explicit, inspectable, and configurable.

2.1 Terminology: "GNN" in this manuscript

In this manuscript, **GNN** refers to the Active Inference Institute's **Generalized Notation Notation** (repository) [Active Inference Institute, 2024], a structured notation for state-space and process models—*not* graph neural networks. COGANT translates source code into this notation, producing program-graph and state-space artifacts (state space, observation modalities, actions/policies, likelihood structure, preferences, time settings, parameterization) that downstream tools can consume. Those downstream tools include graph neural network training pipelines, which can ingest the exported tensor views of the program graph, but such neural networks are a *consumer* of COGANT's output, not its output format. Where the manuscript discusses graph neural networks as related work, it uses the phrase "graph neural networks" in full; the acronym GNN always refers to Generalized Notation Notation.

2.2 Problem

A research or engineering group typically has:

- **Source code** in one or more languages.
- **Analysis goals** that combine classical queries (who calls whom?) with learned models (bug detection, retrieval, summarization).
- **Tooling fragmentation**: compilers and linters produce one shape of facts; PyTorch or JAX expect another.

COGANT unifies these behind a single pipeline whose intermediate artifacts are documented as a progression of IRs (repo IR, program graph, semantic mapping, state space, process model, validation), as described in `../cogant/docs/ARCHITECTURE.md` and `../cogant/docs/SPEC.md`.

2.3 What COGANT is not

COGANT is not a replacement for a full compiler IR, a theorem prover, or a security scanner by itself. It **extracts and serializes** program graphs and behavioral sketches for downstream use. Language coverage and rule depth follow the implementation-status table in the SPEC: Python is the primary front end at v0.1.x; additional languages and Rust acceleration are staged as documented there.

2.4 Manuscript versus package docs

The canonical API and CLI details live in:

- `../cogant/docs/API_GUIDE.md` — Session, `PipelineRunner`, `Bundle`, `ReviewAPI`

- `../cogant/docs/CLI_GUIDE.md` — commands and flags
- `../cogant/docs/GNN_EXPORT.md` — Generalized Notation Notation schema, section ordering, and companion interop field contracts (including optional PyG/DGL tensor views)
- `../cogant/docs/PLUGIN_API.md` — parsers, rules, validators, exporters
- `../cogant/docs/ARCHITECTURE.md` — layers, crates, data flow

This manuscript summarizes theory and practice for readers who want a single linear narrative before diving into those files.

2.5 Dual Python/Rust architecture

COGANT employs a dual-language architecture. The **Python orchestration layer** handles session management, pipeline coordination, configuration, file discovery, AST parsing, and the plugin interface – tasks where developer ergonomics and extensibility matter more than raw throughput. The **Rust core**, organized as a workspace of seven crates (`cogant-core`, `cogant-graph`, `cogant-translate`, `cogant-statespace`, `cogant-gnn`, `cogant-store`, `cogant-ffi`), implements typed graph operations, translation engine internals, state-space structures, and Generalized Notation Notation (GNN) formatting – tasks where memory layout and cache locality dominate wall-clock time. Communication between layers flows through PyO3 bindings (`cogant-ffi`), which release the Python GIL during Rust execution to permit concurrent stage processing. Where Rust bindings are not yet wired for a code path (see the implementation-status table in `../cogant/docs/SPEC.md`), Python fallback implementations ensure the pipeline remains functional at the cost of higher latency on large repositories.

2.6 Roadmap of the following sections

Section 2 formalizes the program graph and IR progression. Section 3 describes the realized pipeline behavior and artifacts. Sections 4–7 cover conclusions, how to run experiments, reproducibility, and related work.

3 Methodology: Program Graphs and Intermediate Representations

3.1 Program graph

Let $G = (V, E)$ denote a directed graph whose vertices V represent program entities and whose edges E represent relationships. COGANT assigns each node a **kind** (for example function, variable, type) and a **semantic role** (for example definition versus use). Each node and edge may carry:

- A **stable identifier** for persistence across runs when hashing allows.
- **Type strings** and attributes recovered from the front end.
- A **confidence** score in $[0, 1]$ and **provenance** metadata (source code, type system, control flow, heuristic, external tool).

This follows the same philosophical line as code property graphs that fuse AST, control flow, and data flow into one analyzable structure [Yamaguchi et al., 2014], but orients the representation toward **Generalized Notation Notation (GNN) export** and, optionally, tensorized views of the same graph: node kinds and roles map to discrete indices in both forms, and optional embeddings can augment features as described in `../cogant/docs/GNN_EXPORT.md`.

3.1.1 Structural equivalence

Two program graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are usefully compared via typed graph isomorphism: a bijection $\phi : V_1 \to V_2$ preserving edge structure and relevant labels supports deduplication and cross-repository linking when interfaces align.

$$(u, v) \in E_1 \iff (\phi(u), \phi(v)) \in E_2 \tag{1}$$

Equation 1 formalizes the structural invariant we check during deduplication and cross-repository linking: a candidate bijection $\phi) is accepted only when every edge in G_1$ has a corresponding edge between the images of its endpoints in G_2 (and vice versa), so that label-preserving matches strictly preserve the adjacency structure of both program graphs.

3.2 Progressive IRs

Processing proceeds through a sequence of representations (see `../cogant/docs/SPEC.md`):

1. **Repo IR** — entities and relationships extracted from parsers.
2. **Program graph IR** — consolidated graph with deduplication and metadata.
3. **Semantic mapping IR** — output of the translation rule engine.
4. **State space IR** — variables, actions, transitions, observations.
5. **Process model IR** — higher-level control patterns where implemented.
6. **Validation IR** — coverage, confidence analysis, schema checks.

Not every stage is equally complete for every repository; the SPEC marks partial areas (translation rules, state space, Rust acceleration) explicitly.

3.3 Translation rules

The **translation** stage applies declarative rules that refine roles, attach labels, and adjust confidence. Concurrency targets and layering are described in `../cogant/docs/ARCHITECTURE.md`. The rule engine composes passes over the graph in a **fixpoint loop**: rules are re-applied until no new semantic mappings emerge or a configurable iteration cap is reached. Overlapping rules may require priority ordering as documented there; when multiple rules claim the same graph fragment, the engine retains the mapping with the highest confidence score, following the principle that edit representations should be composable [Yin et al., 2019].

3.3.1 Fixpoint iteration, conflict resolution, and coverage

The shipped `TranslationEngine` in `../cogant/py/cogant/translate/engine.py` realizes the fixpoint loop concretely. Each iteration walks every registered rule once: the engine calls `rule.matches(graph, query)` to collect candidate fragments, then calls `rule.apply(graph, match)` on each, accumulating any resulting `SemanticMapping` objects keyed by their stable IDs. A per-pass counter tracks mappings that were genuinely new (not already present in the running set), and the loop terminates as soon as a pass completes with zero additions. The engine logs each iteration boundary through an internal match log, so the number of passes required to reach a fixed point is directly observable in the post-run diagnostics. The default iteration cap is `max_iterations = 10`; in testing, most repositories converge well before that bound, and the cap exists primarily as a safety valve against pathological rule sets that could otherwise oscillate indefinitely.

After fixpoint termination, the engine invokes `_resolve_conflicts()` to reconcile mappings whose `graph_fragment_node_ids` sets overlap. For each overlapping pair the engine retains the mapping with the higher `confidence_score` and discards the other, logging a `conflict_resolved` event that records the losing ID, the winning ID, and the specific overlap set. Rule priority is therefore expressed indirectly through the confidence model rather than through a separate ordering table: a higher-priority rule expresses its precedence by yielding mappings with higher scores, which then win in the conflict resolution pass. A companion entry point, `translate_with_confidence()`, runs the standard fixpoint loop, rescores every surviving mapping through the `ConfidenceModel`, and then re-resolves conflicts so that any ordering shifts induced by rescoring are honored.

Translation coverage – the fraction of graph nodes that received at least one semantic mapping – is reported by `get_coverage_report(graph)`. It returns the total node count, the number of covered nodes, the number of uncovered nodes, a `coverage_percent` value rounded to two decimal places, and the sorted list of uncovered node IDs. The uncovered list is intentionally emitted verbatim so that downstream tooling can target unmapped regions for manual review or rule extension, rather than burying the gap behind a single aggregate number [Allamanis et al., 2018].

Algorithm 1 summarizes the engine. Termination is guaranteed because each iteration either produces at least one new mapping (whose stable ID is then fixed in $\mathcal{M}$) or terminates by the break condition; the outer K bound serves as a safety valve for pathological rule sets.

Algorithm 2 detects conflicts via an inverted index in $O(\sum_v |\mathcal{J}(v)|^2)$ worst-case time, which is substantially faster than the naive all-pairs scan for graphs where most nodes carry at most one mapping.

Algorithm 1: Fixpoint translation engine

Input: Program graph $G = (V, E)$; rule set R; maximum iterations K (default 10)
Output: Set of semantic mappings $\mathcal{M}$ keyed by stable ID

$\mathcal{M} \leftarrow \emptyset$;
for $k \leftarrow 1$ **to** K **do**
  $n_{\text{new}} \leftarrow 0$;
  foreach *rule* $r \in R$ *sorted by priority*(r) *descending* **do**
    foreach *match* $m \in r.matches(G)$ **do**
      $\mu \leftarrow r.\text{apply}(G, m)$;
      if $\mu \neq \bot$ ***and*** $\mu.id \notin \mathcal{M}$ **then**
        $\mathcal{M} \leftarrow \mathcal{M} \cup \{\mu\}$;
        $n_{\text{new}} \leftarrow n_{\text{new}} + 1$;
  if $n_{new} = 0$ **then**
    break ; `// fixed point reached`
$\mathcal{M} \leftarrow \textsc{ResolveConflicts}(\mathcal{M})$;
return $\mathcal{M}$;

Algorithm 2: Priority-ordered conflict resolution

Input: Mapping set $\mathcal{M}$; priority function $p(\cdot)$
Output: Reduced mapping set with no overlapping fragments

Build inverted index $\mathcal{I} : V \to 2^{\mathcal{M}}$ from node IDs to mappings touching them;
$\mathcal{C} \leftarrow \emptyset$; `// conflict pairs`
foreach *node* v *with* $|\mathcal{I}(v)| \geq 2$ **do**
  foreach $(\mu_a, \mu_b) \in \binom{\mathcal{I}(v)}{2}$ **do**
    $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\mu_a, \mu_b)\}$;
$\mathcal{R} \leftarrow \emptyset$; `// removal set`
foreach $(\mu_a, \mu_b) \in \mathcal{C}$ **do**
  if $\mu_a \in \mathcal{R}$ ***or*** $\mu_b \in \mathcal{R}$ **then**
    continue
  $k_a \leftarrow (p(\mu_a), c(\mu_a))$; $k_b \leftarrow (p(\mu_b), c(\mu_b))$;
  if $k_a \geq k_b$ **then**
    $\mathcal{R} \leftarrow \mathcal{R} \cup \{\mu_b\}$
  else
    $\mathcal{R} \leftarrow \mathcal{R} \cup \{\mu_a\}$
return $\mathcal{M} \setminus \mathcal{R}$;

3.4 Confidence Scoring and Evidence Tiers

The shipped `ConfidenceModel` in `../cogant/py/cogant/translate/confidence.py` computes a scalar score in $[0, 1]$ from:

- Average confidence over provenance records.
- An evidence-diversity term (scaled, capped).
- A parser certainty factor applied multiplicatively.
- Conflict penalties subtracted after scaling.

$$c = \max\left(0,\ \min\left(1,\ (\bar{e} + \delta_d) \cdot \kappa - \pi\right)\right) \tag{2}$$

Here $\bar{e}$ is the mean evidence confidence, δ_d is the diversity bonus (bounded), $\kappa) is parser certainty, and \pi) aggregates conflic only versus static-plus-runtime) are assigned from score thresholds and evidence source tags (`determine_confidence_ti`

3.5 State space and behavior

Where traces or coverage are available, **dynamic** extraction feeds the state-space compiler. The goal is a compact behavioral model: states, actions, transitions, and observations that sit alongside the static graph for tasks that require execution-sensitive features.

The shipped `StateSpaceCompiler` in `../cogant/py/cogant/statespace/compiler.py` constructs this model in several coordinated passes driven by the semantic mappings and the underlying program graph.

State variables are identified by a `StateVariableExtractor` that traverses graph nodes carrying READS and WRITES edges: any node that participates in a write is a candidate hidden-state variable, and its type, cardinality, and initial confidence are derived from the node's recovered type string and the provenance of the rules that produced the associated mapping.

Actions are extracted from semantic mappings of kind `ACTION`, with parameters read from node metadata (typically the parser-recovered function signature), effects traced through outgoing WRITES edges on the controller node, and preconditions derived from parameter lists, docstring directives, and explicit metadata entries. For method-kind actions, the compiler also walks CONTAINS edges back to the enclosing class so that instance-level state mutations are attributed to the correct controller, mirroring how code property graphs fuse structural and data-flow views [Yamaguchi et al., 2014].

Transitions are inferred by a cross-reference pass (`_cross_reference_actions_and_variables`) that, for each action, partitions its adjacent variables into reads and writes based on edge kind (WRITES and MUTATES yielding writes; READS and OBSERVES yielding reads). The resulting `Transition` object records a `source_state` in which every touched variable is marked `"pre"` and a `target_state` in which written variables advance to `"post"` while read-only variables remain `"pre"`; this simple pre/post convention keeps the model faithful to the static evidence without overcommitting to symbolic value domains that cannot be recovered from AST analysis alone. Trigger attribution follows incoming TRIGGERS and CALLS edges so that orchestration flow is preserved in the behavioral model.

Observation modalities are built from semantic mappings of kind `OBSERVATION`: each associated node becomes an `ObservationModality` whose modality type (`log`, `metric`, `event`, `sensor`, or a generic fallback) is inferred from the mapping's description and the node's name, giving the GNN export a typed observation channel aligned with the OBSERVES edges in the program graph.

The **temporal regime** of the model is determined by a companion `TemporalAnalyzer`. It classifies nodes as asynchronous when their metadata carries `is_async`/`async` flags or their names match patterns such as `async`, `callback`, `promise`, or `future`, and as event-related when their kind is `EVENT` or their names match `event`, `handler`, `listener`, or `trigger`. Temporal orderings are then extracted from CALLS and TRIGGERS edges, with each edge classified as `parallel` (when either endpoint is async) or `sequential` otherwise, and event patterns are assembled from triggers-in / triggers-out pairs around each event node. A final decision rule selects among `SYNCHRONOUS`, `ASYNCHRONOUS`, `EVENT_DRIVEN`, and `HYBRID`: the presence of event triggers and patterns combined with async handlers yields `HYBRID`; event triggers alone yield `EVENT_DRIVEN`; an async fraction above 30 percent or the presence of async handlers yields `ASYNCHRONOUS`; and all remaining cases default to `SYNCHRONOUS`. This regime is attached to the `StateSpaceModel` metadata so downstream consumers know which execution model the transition graph assumes.

When coverage and traces are available, `enrich_graph()` in `../cogant/py/cogant/dynamic/enrichment.py` feeds additional evidence into this pipeline. Coverage enrichment matches `.coverage` SQLite databases or Cobertura XML reports against nodes whose `path` and `source_range` overlap covered lines, attaching `coverage_hits` and, where branch data is available, `branch_coverage` metadata. Trace enrichment parses Chrome DevTools traces, writes `call_count`, `avg_duration_ms`, and `is_hot_path` onto matching callable nodes, and adds or reweights dynamic CALLS edges tagged with `evidence_sources=["dynamic_trace"]`. Both steps also append `dynamic_coverage` and `dynamic_trace` markers to the program graph's evidence sources. The confidence model consumes these markers directly: any mapping whose evidence set now contains both static and dynamic entries becomes eligible for promotion from the `STATIC_ONLY` tier to `STATIC_PLUS_RUNTIME`, and hot-path and branch-coverage metadata raise the underlying score through the diversity bonus δ_d in Equation 2. This is the mechanism by which executing the target program, even partially, converts static heuristics into corroborated behavioral facts without rerunning the upstream rule engine [Allamanis et al., 2018].

3.6 GNN export (Generalized Notation Notation)

Exports package the program graph and compiled state-space model into the Active Inference Institute's **Generalized Notation Notation** plus a small set of interop targets:

- **GNN Markdown (`model.gnn.md`)** — canonical, human-readable Generalized Notation Notation organized into the section order defined in `../cogant/docs/GNN_EXPORT.md` (Model Metadata, Repository Metadata, Source Coverage, State Space, Observation Modalities, Actions/Policies, Connections, Factors, Transition Structure, Likelihood Structure, Preferences/Constraints, Time Settings, Parameterization, Ontology Mapping, Provenance, Confidence Scores, Rendering Hints, Validation Notes).
- **GNN JSON (`model.gnn.json`)** — machine-readable equivalent of the markdown sections, plus companion JSON files per section (`state_space.json`, `observations.json`, `actions.json`, `transitions.json`, and so on).

- **Interop exports** — GraphML and Parquet for analysis in Gephi/yEd and DuckDB, and optional tensor views (PyTorch Geometric `Data` objects, DGL graphs, HDF5 tables) for downstream graph neural network training pipelines that consume the program graph as a relational tensor.

Node and edge feature breakdowns, section contracts, and the optional framework targets are specified in `../cogant/docs/GNN_EXPORT.md`; they determine the structure of the emitted notation as well as the effective input dimensionality of any downstream model.

3.7 Error handling philosophy

The implementation distinguishes fatal configuration or parse failures from per-file errors that allow partial results. Validation aggregates warnings and inconsistencies into a report that travels with the bundle. This mirrors the layered error categories documented in the architecture (fatal, error, warning, info).

4 API and Workflows

This section describes the programmatic surface that the shipped Python package exposes in practice, aligned with `../cogant/docs/API_GUIDE.md` and `../cogant/VERIFICATION_REPORT.md`. It is **not** an empirical benchmark section; COGANT does not ship comparative timing claims in the manuscript layer. Instead, it records the **surface area** users can rely on: two complementary entry points (a Session for stepwise work and a Pipeline for batch runs), the Bundle accessors that expose their artifacts, a command-line interface, and a Review API for human-in-the-loop curation.

4.1 Session-oriented workflow

The Session API is intended for interactive exploration: it is the right choice when a user wants to inspect intermediate artifacts between stages, iterate on a single repository in a notebook, or debug a specific extraction step without committing to a full scripted run. Each call returns control to the caller so that the graph, mappings, or state-space model can be examined before the next stage is invoked.

`Session.from_target` accepts a local path or URL, then supports a stepwise workflow:

- `extract_static` — AST-oriented extraction for supported languages.
- `extract_dynamic` — traces and coverage when inputs exist.
- `build_graph` — program graph construction.
- `translate_to_gnn` — Generalized Notation Notation (GNN) representation.
- `compile_state_space` — behavioral model when the pipeline has sufficient data.
- `export_all` — writes JSON artifacts under a chosen output directory.

This path suits interactive notebooks and incremental debugging.

4.2 Pipeline-oriented workflow

The Pipeline API is intended for scripted, reproducible batch runs: it is the right choice when a user wants to process many repositories with a fixed configuration, wire COGANT into CI, or guarantee that every run executes the same ordered stages with the same plugin settings. Because the whole run is described by a single `PipelineConfig`, it can be checked into version control and replayed without manual intervention.

`PipelineRunner` with `PipelineConfig` runs an ordered list of stages (ingest, static, normalize, graph, translate, state space, process, export, validate). Configuration can skip stages (for example dynamic analysis), attach **plugin** settings per language, set `output_dir`, verbosity, and dry-run mode. Results aggregate into a **Bundle** with `stage_results`, error lists, and accessors described below.

4.3 Bundle accessors

The bundle API exposes summaries and artifacts, including:

- `repo_summary`, `program_graph`, `state_space_model`, `process_model`
- `gnn_markdown` — human-readable graph summary
- `validation_report`

- `render_site` — static HTML site with graph and model views
- `to_json` / `save_json`

The verification report lists these as delivered capabilities for the v0.1.0 line.

4.4 Command-line interface

The CLI entry point (`cogant.cli.main`) implements commands such as `init`, `scan`, `extract-static`, `extract-dynamic`, graph and export verbs, and validation-oriented subcommands. Exact flags live in `../cogant/docs/CLI_GUIDE.md`; the manuscript does not duplicate them to avoid drift.

4.5 Review API

`ReviewAPI` supports interactive curation: load a bundle, present mappings, accept, reject, or edit, then save a curated bundle. This closes the loop when human review is part of the ML dataset construction.

5 Examples and Failure Modes

This section walks through a concrete example run of the pipeline on a small Flask application, shows representative fragments of the artifacts it produces (semantic mappings and state-space transitions), and then documents the degradation behavior that users should expect when inputs are missing or partial. Together these illustrate both what a successful run looks like and how the system communicates partial success.

5.1 Example outputs

The package tree includes examples under `../cogant/examples/` and sample exports under `../cogant/output/examples/control_positive/` (for example **calculator**, **event_pipeline**, and **flask_mini** with **model.gnn.json** / **model.gnn.md** under `data/` and `reports/`, diagrams under `diagrams/`, and raster figures under `figures/`). Use `cogant translate --layout-output` or `PipelineConfig(layout_output=True)` to place pipeline JSON under `data/` automatically; run `python -m cogant.tools.render_output_figures` on the output root for PNGs. Regenerate these trees by running the packaged pipeline against the example sources when validating releases.

5.1.1 Concrete walkthrough: Flask REST API

To make the pipeline's behavior tangible, consider a small Flask application with three HTTP endpoints, two utility modules, and one data-access layer – 800 lines of Python across 6 files, of which 782 lines are non-blank, non-comment code analyzed by the pipeline. Running `cogant scan --target ./flask_app && cogant extract-static && cogant build-graph && cogant translate && cogant export-all -o output/` on this repository produces a program graph with the following characteristics:

Metric	Value
Source files discovered	6
Lines analyzed	782 / 800 (97.8%)
Nodes	**147**
Edges	**389**
Mean node confidence	0.91
Validation status	PASS (0 errors, 3 warnings)

The node and edge populations distribute across kinds as follows:

Table 1. Node kind distribution (Flask API example).

Node kind	Count	Percentage
FUNCTION	38	25.9%
VARIABLE	52	35.4%
TYPE	11	7.5%
MODULE	6	4.1%
CONTROLFLOW_NODE	18	12.2%

Node kind	Count	Percentage
DATA_STRUCTURE	7	4.8%
ERRORHANDLER	5	3.4%
CONSTANT	6	4.1%
EXTERNAL	4	2.7%

Table 2. Edge kind distribution (Flask API example).

Edge kind	Count	Percentage
CALLS	87	22.4%
USES	104	26.7%
DEFINES	62	15.9%
HAS_TYPE	48	12.3%
DATA_FLOW	34	8.7%
MEMBER_OF	29	7.5%
INHERITS	3	0.8%
Other	22	5.7%

5.1.2 Example semantic mapping output

During the translation stage, each program-graph node is matched against the active rule set and assigned a semantic role with an associated confidence score. The following excerpt shows three representative mappings from the Flask example:

```
Node: get_users  (kind=FUNCTION, file=routes/users.py:14)
  Matched rule: rule_fn_def_001
  Target role:  FUNCTION_DEF
  Confidence:   0.98  (base=1.0, -0.02 missing docstring penalty)
  Provenance:   SourceCode

Node: db.session.query  (kind=FUNCTION, file=routes/users.py:22)
  Matched rule: rule_method_call_001
  Target role:  METHOD_CALL
  Confidence:   0.82  (base=0.90, -0.08 receiver type inferred heuristically)
  Provenance:   Heuristic

Node: User  (kind=TYPE, file=models/user.py:5)
  Matched rule: rule_type_def_001
  Target role:  TYPE_DEF
  Confidence:   1.00
  Provenance:   SourceCode
```

The `db.session.query` call illustrates how the confidence model (Equation 2 in Section 2) penalizes heuristic provenance: the receiver type of `session` is resolved by import tracing rather than explicit annotation, reducing $\kappa) below 1.0 and yielding a final confidence of 0.82 in the MEDIUM tier$.

5.1.3 Example state-space excerpt

When dynamic traces are available, the state-space compiler produces a behavioral model alongside the static graph. The following excerpt shows a fragment of the state-space IR for the `/users` endpoint:


```
{
  "variables": [
    {"name": "request.method", "type": "str", "domain": ["GET", "POST"]},
    {"name": "db_connected",   "type": "bool", "domain": [true, false]},
    {"name": "response_code",  "type": "int",  "domain": [200, 400, 500]}
  ],
  "actions": [
    {"name": "validate_input", "source": "routes/users.py:18"},
    {"name": "query_database",  "source": "routes/users.py:22"},
    {"name": "format_response", "source": "routes/users.py:30"}
  ],
  "transitions": [
    {
      "from_state": {"request.method": "GET", "db_connected": true},
      "action": "query_database",
      "to_state": {"response_code": 200},
      "confidence": 0.94,
      "tier": "STATIC_PLUS_RUNTIME"
    },
    {
      "from_state": {"request.method": "POST", "db_connected": true},
      "action": "validate_input",
      "to_state": {"response_code": 400},
      "confidence": 0.71,
      "tier": "STATIC_ONLY"
    },
    {
      "from_state": {"db_connected": false},
      "action": "query_database",
      "to_state": {"response_code": 500},
      "confidence": 0.58,
      "tier": "RUNTIME_ONLY"
    }
  ]
}
```


Confidence tiers follow the thresholds defined in `determine_confidence_tier`: **STATIC_PLUS_RUNTIME** ($c \geq 0.65$, with both static and dynamic evidence) indicates corroboration from AST analysis and execution traces; **STATIC_ONLY** ($c \geq 0.5$, static evidence only) reflects assertions grounded in source structure alone; **RUNTIME_ONLY** ($c \geq 0.4$, dynamic evidence only) flags inferences from runtime data without static corroboration. A fourth tier, **HUMAN_REVIEWED** ($c \geq 0.9$, with human review evidence), is available for manually curated mappings.

5.1.4 Failure modes and graceful degradation

COGANT is designed so that missing or partial inputs degrade the output bundle rather than halt it, and the manuscript records these degradation paths explicitly so that downstream consumers can interpret a partial run without guessing what was skipped [Peng, 2011]. Five failure modes are worth naming because each has a visible signature in the emitted artifacts.

No dynamic traces available. When neither coverage data nor execution traces are supplied, `enrich_graph()` is a no-op: no `coverage_hits`, `branch_coverage`, `call_count`, `avg_duration_ms`, or `is_hot_path` metadata is attached, and the graph's `evidence_sources` list never acquires `dynamic_coverage` or `dynamic_trace` markers. The state-space compiler still runs end-to-end using the semantic mappings and static graph alone, but every resulting transition is confined to the `STATIC_ONLY` tier (or lower), and no transition can be promoted to `STATIC_PLUS_RUNTIME`. Downstream code that consumes the bundle can identify a purely-static run at a glance by checking the evidence source markers on the graph metadata rather than scanning individual transitions.

Incomplete coverage data. When coverage is supplied but covers only a subset of the project, `_enrich_with_coverage` annotates the matched files only. The enrichment loop walks each coverage span, normalizes the reported file path, looks up candidate nodes whose path matches, and annotates those whose `source_range` overlaps the covered line; unmatched files are silently skipped with an informational log line, and nodes in unmatched files retain no coverage metadata at all. The result is a partially enriched graph in which some regions are eligible for tier promotion and others are not. Because the enrichment summary returns the number of annotated nodes, users can check whether the observed annotation rate matches their expectations for the provided coverage input.

Translation rules that do not match. When the active rule set fails to produce any mapping for a node – whether because no rule fires, or because all firing rules are pruned during conflict resolution – that node remains outside `self.mappings` and simply does not appear in any `graph_fragment_node_ids`. The translation engine's `get_coverage_report()` reports this directly: the returned dictionary contains the total node count, the covered count, a two-decimal `coverage_percent`, and the sorted list of `uncovered_node_ids`. Rather than treat uncovered nodes as an error, the engine surfaces them as an explicit gap list so that authors can either extend the rule set, hand the gap list to the `ReviewAPI` for curation, or accept the partial mapping for downstream Generalized Notation Notation (GNN) export.

Fixpoint non-convergence. The translation engine bounds its fixpoint loop at `max_iterations = 10` by default. If a rule set is pathological enough to keep emitting new mappings past that bound – for example because two rules can produce mutually triggering mappings – the engine emits a `"Max iterations reached without convergence"` warning, stops the loop, and proceeds to conflict resolution with whatever mappings it has accumulated. The iteration cap therefore guarantees termination even for misconfigured rule sets, and the warning in the log gives rule authors a clear signal that the cap was hit. Because each iteration also writes an `iteration_complete` entry into the engine's internal match log, the exact per-pass mapping counts are available for diagnosis without rerunning the pipeline.

Pipeline error tolerance. The default pipeline configuration (`cogant/py/cogant/config/defaults.py`) marks the `dynamic`, `translate`, `state_space`, and `process` stages with `skip_on_error=True`,

while structural stages such as `ingest`, `static`, `graph`, `export`, and `validate` keep the stricter default `skip_on_error=False`. When an optional stage raises, the runner records the error in the bundle, emits a warning, and continues to the next stage rather than aborting the run. Downstream bundle accessors (`state_space_model`, `process_model`) simply return `None` for stages that did not produce output, so a downstream consumer can detect a skipped stage by checking its accessor rather than by parsing log text. The net effect is that a partial bundle – for example, a program graph and semantic mappings without a state-space model – remains a first-class artifact with a clear provenance trail, rather than an opaque failure.

5.2 Rust layer

Native crates (`cogant-core`, `cogant-graph`, `cogant-translate`, `cogant-statespace`, `cogant-gnn`, `cogant-ffi`, and related packages) implement typed graph operations and export formatting. When PyO3 bindings are active, Python delegates heavy graph work through `cogant-ffi`. Where bindings are not yet wired for a code path, Python fallbacks apply; see SPEC **Implementation status** for the current boundary.

6 Conclusion

COGANT frames codebase analysis as a pipeline from ingestion through a program graph IR to **Generalized Notation Notation (GNN)** exports, with explicit confidence and provenance so that learning systems can treat analysis noise as data rather than as hidden failure modes. The Python layer provides session and pipeline APIs, a bundle abstraction, CLI, review tooling, and HTML reporting; the Rust layer concentrates graph mechanics and export formatting under crate boundaries described in `../cogant/docs/ARCHITECTURE.md`.

6.1 Shipped Capabilities

Several capabilities ship in the current v0.1.x line and together define the behaviour that downstream users can rely on:

1. **Fixpoint translation engine.** The shipped `TranslationEngine` re-applies every registered rule on each pass, terminates as soon as a pass produces zero new mappings, and bounds pathological rule sets with a configurable iteration cap. Each iteration boundary is recorded in the internal match log, so convergence can be audited after the fact.
2. **Rule priority and conflict resolution.** Overlapping mappings are reconciled by confidence score: when two mappings cover overlapping `graph_fragment_node_ids`, the engine retains the higher-confidence mapping and records a `conflict_resolved` event naming the winner, the loser, and the overlap. Rule priority is therefore expressed through the confidence model rather than a separate ordering table, and `translate_with_confidence()` re-scores and re-resolves so that priority shifts from the `ConfidenceModel` are honoured.
3. **Dynamic enrichment from coverage and traces.** When `.coverage` SQLite or Cobertura XML inputs and Chrome DevTools traces are supplied, `enrich_graph()` annotates nodes with `coverage_hits`, `branch_coverage`, `call_count`, `avg_duration_ms`, and `is_hot_path` metadata, and appends `dynamic_coverage` and `dynamic_trace` markers to the program graph's evidence sources.
4. **Confidence tier promotion.** Mappings whose evidence set acquires dynamic markers become eligible for promotion from `STATIC_ONLY` to `STATIC_PLUS_RUNTIME`, and hot-path and branch-coverage metadata raise the underlying score through the diversity bonus in Equation 2. A purely-static run remains a first-class bundle; it is simply marked as such on the graph metadata.
5. **Ten-stage pipeline architecture.** The runner executes an ordered DAG (ingest, static, normalize, graph, translate, state space, process, export, validate, and report), with `skip_on_error=True` on optional stages so that a missing dynamic input or a pathological rule set produces a partial but well-formed Generalized Notation Notation bundle rather than an opaque failure.

Limitations follow the honest scope in `../cogant/docs/SPEC.md`: multi-language support beyond Python is largely roadmap; translation rules and state-space extraction are **partial** and repository-dependent; native acceleration is staged. Users should validate exports on their own corpora before trusting downstream model metrics.

Intended users include researchers building datasets from open-source repositories, teams prototyping Active Inference models or graph neural network training pipelines over program-graph data who need a single export contract, and engineers extending the system via

`../cogant/docs/PLUGIN_API.md`.

Validation in the software-engineering sense is split: the repository's verification report enumerates implemented modules and entry points; scientific validation of model quality remains the responsibility of downstream training and evaluation code.

6.2 Roadmap and Future Extensions

Several concrete directions extend the current system:

1. **Multi-language parsers.** The v0.1.x front end targets Python; adding parsers for JavaScript/TypeScript, Java, and Go would cover the majority of open-source ML-relevant repositories. Each parser implements the plugin interface documented in `../cogant/docs/PLUGIN_API.md`, so language additions do not require changes to the core IR or export pipeline.

2. **Rust acceleration of critical paths.** The native crate layer (`cogant-graph`, `cogant-translate`) already defines typed graph operations. Wiring PyO3 bindings for the hot paths — deduplication, rule matching, and Generalized Notation Notation section/tensor packing in `cogant-gnn` — would reduce end-to-end latency for large repositories from minutes to seconds, based on preliminary profiling of the Python fallback implementations.

3. **Incremental re-analysis.** Currently the pipeline processes a full repository snapshot on each invocation. An incremental mode that accepts a Git diff and updates only affected subgraphs would enable integration into CI/CD workflows where per-commit turnaround matters. The stable-identifier scheme already supports cross-run node matching; the missing piece is a dependency tracker that invalidates downstream IRs when upstream nodes change.

4. **LLM-assisted rule discovery.** Translation rules are currently hand-authored. A semi-automated workflow could present a large language model with unannotated graph fragments and ask it to propose candidate rules, which a human reviewer then accepts, edits, or rejects via the existing `ReviewAPI`. This combines the pattern-recognition strength of LLMs with the auditability of declarative rules.

5. **Cross-repository graph linking.** When multiple repositories share interfaces (for example, a library and its consumers), linking their program graphs at call boundaries produces a richer training signal for tasks such as API misuse detection and cross-project code search. The graph homomorphism property defined in Section 2 provides the formal basis for identifying shared interface nodes across independently analyzed repositories.

7 Experimental setup

7.1 Environment

Requirements: Python 3.11 or newer (enforced in `pyproject.toml`), plus an optional Rust toolchain (`cargo`, stable 1.70+) when building native acceleration crates under `../cogant/rust/`.

From the COGANT package root `../cogant/` (where `pyproject.toml` and `py/cogant/` live), install with `uv sync --all-extras`, or `pip install -e ".[dev,viz]"` / `pip install -e ".[all]"` as in GETTING_STARTED.md. Run those commands from that directory when working inside this monorepo layout. Python sources live under `../cogant/py/cogant/`; see the package README.md.

7.2 Running the API

Minimal **Session** run:

```
from cogant import Session

session = Session.from_target("./path/to/repo")
session.extract_static()
session.build_graph()
session.translate_to_gnn()
session.export_all("output/")
```

Minimal **Pipeline** run:

```
from cogant import PipelineRunner
from cogant.api.pipeline import PipelineConfig

runner = PipelineRunner()
config = PipelineConfig(output_dir="output/")
bundle = runner.run("./path/to/repo", config)
```

Adjust `PipelineConfig.stages`, `skip_stages`, and `plugins` to match the languages and tooling available on the machine.

7.3 Configuration files

YAML configuration can drive pipeline behavior (paths, stages, plugin options). The architecture and SPEC documents describe the configuration surface; keep project-specific secrets out of version control.

A minimal pipeline configuration looks like this:

```
# cogant.config.yaml
stages:
  - ingest
  - static
  - normalize
```

```yaml
  - graph
  - dynamic       # optional; skipped if no coverage/trace inputs
  - translate
  - statespace
  - process
  - export
  - validate

skip_stages: []   # e.g. ["dynamic"] to force static-only runs

plugins:
  dynamic:
    coverage_path: "./coverage.xml"
    trace_path: "./chrome_trace.json"
    hot_path_percentile: 10
  translate:
    max_iterations: 10
    static_only_threshold: 0.5
    static_plus_runtime_threshold: 0.65

output_dir: "./cogant_output/"
verbose: true
dry_run: false
```

Each stage key corresponds to a handler in `cogant.api.pipeline.PipelineRunner.stage_handlers`; plugin sub-dictionaries are passed through to the stage at invocation time. The threshold keys under `plugins.translate` mirror the constants defined in `cogant.translate.confidence.ConfidenceModel`, keeping the YAML surface and the Python defaults in lockstep.

7.4 CLI

Use the `cogant` CLI for scripted batch runs—see `../cogant/docs/CLI_GUIDE.md` for the command list that matches the installed version.

7.5 Export targets

The primary export targets are the **Generalized Notation Notation (GNN)** canonical Markdown (`model.gnn.md`) and the equivalent companion JSON files described in `../cogant/docs/GNN_EXPORT.md`. Optional interop targets (GraphML, Parquet) support analysis in Gephi/yEd and DuckDB, and optional tensor views for PyTorch Geometric, DGL, or HDF5 can be selected when downstream graph neural network training pipelines need to consume the program graph as a relational tensor. Ensure the Python environment includes optional dependencies for these tensor exports when those code paths are used.

7.6 Python AST parser capabilities

The v0.1.x front end relies on `cogant.static.parser.PythonASTParser`, which processes Python source through the standard-library `ast` module at the CPython version available in the runtime (3.9+ recommended). The parser extracts the following construct categories:

- **Module-level entities**: module docstrings, `__all__` exports, top-level assignments.
- **Functions and methods**: `def` and `async def`, including signatures with positional, keyword, variadic (`*args`, `**kwargs`), and positional-only parameters. Default values are recorded as constant expressions where statically evaluable.
- **Classes**: class definitions, base classes, metaclasses, and the `__init__` / `__new__` boundary.
- **Decorators**: `@staticmethod`, `@classmethod`, `@property`, `@dataclass`, and arbitrary user-defined decorators. Decorator arguments are captured as attribute metadata.
- **Type annotations**: PEP 484 / 526 / 604 annotations on function parameters, return types, and variable assignments. Generic subscripts (`List[int]`, `Dict[str, Any]`) are preserved as type strings.
- **Comprehensions and generators**: list, set, dict comprehensions and generator expressions are represented as anonymous FUNCTION nodes with DATA_FLOW edges to their enclosing scope.
- **Control flow**: `if/elif/else`, `for/while/else`, `try/except/finally`, `with/async with`, and `match/case` (Python 3.10+) are mapped to CONTROLFLOW_NODE entities.
- **Imports**: `import` and `from ... import` statements produce MODULE_IMPORT roles with edges to the resolved module when discoverable on the file system.
- **Constants**: module-level and class-level assignments to `Final` or ALL_CAPS names are classified as CONSTANT nodes.

Constructs that require runtime evaluation (for example `exec`, `importlib.import_module`, or dynamic `__getattr__`) are recorded as EXTERNAL nodes with HEURISTIC provenance and correspondingly lower confidence.

7.7 Progressive IR stages

Processing advances through six intermediate representations, each adding semantic detail atop its predecessor. Table 3 summarizes what each stage contributes.

Table 3. Progressive IR stages and their contributions.

Stage	IR name	Key additions	Typical output size (10K-function repo)
1	Repo IR	Raw entities and relationships per file; deduplication; merged type info	~15 MB JSON

Stage	IR name	Key additions	Typical output size (10K-function repo)
2	Program Graph IR	Consolidated directed graph $G = (V, E)$; stable identifiers; confidence and provenance on every node and edge	~20 MB JSON
3	Semantic Mapping IR	Translation rules applied; semantic roles assigned; confidence adjusted by rule engine	~22 MB JSON (graph + mapping log)
4	State Space IR	Variables, actions, transitions, observations; dynamic traces integrated where available	~5 MB JSON (behavioral model)
5	Process Model IR	Higher-level control patterns (request–response, producer–consumer, state machines)	~2 MB JSON
6	Validation IR	Coverage metrics, confidence distribution, schema compliance, consistency checks, reproducibility hashes	~1 MB JSON (report)

Stages 4 and 5 are **partial** for many repositories: the state-space compiler requires either execution traces or sufficient static structure (for example annotated state machines) to produce meaningful output. The pipeline tolerates missing stages gracefully; the Validation IR records which stages completed and which were skipped.

7.8 Performance characteristics

The architecture targets the following benchmarks on a 4-core machine, as specified in `../cogant/docs/ARCHITECTURE.md`:

Repository size	Target wall-clock time	Memory budget
10K functions	< 30 s	< 500 MB
100K functions	< 5 min	< 2 GB
1M functions	< 1 hr	< 2 GB (streaming)

These targets assume the Python orchestration layer with Rust acceleration on critical paths (graph construction, rule matching, and Generalized Notation Notation section/tensor packing in `cogant-gnn`). In the current v0.1.x release, where Rust bindings are staged rather than fully wired, Python fallback implementations handle graph operations; users should expect roughly 2–3× slower throughput on repositories above 50K functions until native acceleration is enabled.

Parallelism is exploited at multiple stages: file parsing is embarrassingly parallel, translation rules are applied concurrently over independent subgraphs, and export formatting runs in parallel per output format. Incremental caching skips unchanged stages when the input hash matches a prior run, which is particularly effective during iterative development against a single repository.

7.9 What to record

For reproducible experiments, record: COGANT version or commit hash, interpreter version, list of stages executed, configuration file contents (redacted), input repository commit hash, and random seeds for any learned components **outside** COGANT that consume the exports.

8 Reproducibility

Reproducible computational research requires version-pinned tools, fixed inputs, and documented outputs [Peng, 2011]. COGANT contributes the **graph-generation** slice of that story: identical source trees and identical pipeline configuration should yield identical exported bundles modulo declared nondeterminism (for example optional neural embeddings if enabled).

8.1 Version pinning

Pin:

- The **COGANT** version (`pyproject.toml` / package `__version__`) and Git commit when installing from source.
- The **Python** minor version.
- **Optional** Rust toolchain commit when building native extensions from `../cogant/rust/`.

8.2 Artifact layout

Pipeline output typically includes JSON for intermediate IRs, **Generalized Notation Notation (GNN)** bundles (canonical `model.gnn.md` plus companion JSON and interop artifacts), validation reports, and optional HTML sites. Paths under the chosen `output_dir` should be treated as disposable but **checksumable**: store hashes alongside published datasets derived from COGANT exports when releasing research artifacts.

8.3 Determinism

Parsing and graph construction aim for deterministic ordering on a fixed filesystem snapshot. Features that pull in external models (for example optional name or documentation embeddings consumed by the Generalized Notation Notation exporter) introduce variability unless models and seeds are fixed; the Generalized Notation Notation export document (`../cogant/docs/GNN_EXPORT.md`) calls out embedding dimensions and optional behavior.

8.4 Relation to the template repository

While this project remains outside `../../../projects/`, it is **not** executed by the root `./run.sh` discovery layer. After promotion to `../../../projects/cogant/` with `src/`, `tests/`, and `pyproject.toml` per template rules, the standard manuscript validation and PDF stages apply; until then, validate Markdown from the template repository root, e.g. `uv run python -m infrastructure.validation.cli markdown ./projects_in_progress/cogant/manuscript/`.

8.5 Validation gates

The pipeline enforces quality through three complementary validation checkers, each targeting a different failure mode:

IntegrityChecker. Verifies structural soundness of the program graph: all edge endpoints reference existing nodes, no unintended duplicate nodes exist, orphaned nodes (zero in-degree and zero out-degree) are flagged, and self-loops are reported unless explicitly allowed by configuration.

The checker also ensures that confidence scores fall within $[0, 1]$ and that provenance records are non-empty for every node and edge. A graph that fails integrity checks receives a FAIL validation status; downstream export is blocked.

SchemaChecker. Validates each IR artifact against its JSON schema (versioned alongside the COGANT package). Schema violations – such as missing required fields, incorrect types, or unknown enum values in `NodeKind` or `SemanticRole` – are classified as ERROR or FATAL depending on severity. Schema versions are recorded in the validation report so that consumers can verify compatibility with their import code.

ProvenanceChecker. Audits the provenance chain: every assertion in the semantic mapping must trace back to at least one evidence source (SourceCode, TypeSystem, ControlFlow, Heuristic, or External). The checker flags mappings whose provenance is empty or whose confidence score is inconsistent with the declared evidence tier – for example, a STATIC_PLUS_RUNTIME tier with no runtime trace evidence. These flags appear as warnings rather than errors, since partial provenance is expected for heuristic rules.

Together, these gates ensure that exported bundles meet a minimum quality bar before reaching downstream models. The thresholds are configurable (see `../cogant/docs/VALIDATION.md`); defaults require $\geq 95\%$ coverage, mean confidence ≥ 0.85, and zero schema violations.

8.6 Pipeline checkpoint and resume behavior

The pipeline writes a checkpoint file after each successfully completed stage. The checkpoint records the stage name, completion timestamp, input hash, and output hash. On a subsequent invocation with `--resume` (or `PipelineConfig.resume = True`), the runner reads the checkpoint file, verifies that the input hash for each completed stage still matches the current source tree, and skips stages whose outputs are already valid. If a source file has changed, the runner invalidates the affected stage and all downstream stages, then re-executes from the earliest invalidated point.

Checkpoints are stored as JSON under the configured `output_dir`:

```
output/
  .cogant_checkpoint.json
```

This mechanism is particularly useful during iterative development: after an initial full run, changing a single source file triggers only re-parsing, graph construction, and downstream stages for the affected subgraph, rather than re-analyzing the entire repository.

8.7 Output manifest structure

Each pipeline run produces a manifest file (`manifest.json`) alongside the exported artifacts. The manifest provides a machine-readable inventory of everything the run produced:

```
{
  "cogant_version": "0.1.0",
  "run_id": "run_20260408_143012",
  "timestamp": "2026-04-08T14:30:12Z",
  "input_hash": "sha256:a1b2c3...",
  "config_hash": "sha256:d4e5f6...",
```

```
  "stages_completed": ["discovery", "parsing", "repo_ir", "graph",
                       "translate", "validate", "export"],
  "stages_skipped": ["statespace", "process"],
  "artifacts": [
    {"path": "model.gnn.json", "format": "json", "size_bytes": 2048576,
     "hash": "sha256:..."},
    {"path": "model.pyg.pt",   "format": "pytorch_geometric", "size_bytes": 1024000,
     "hash": "sha256:..."},
    {"path": "validation_report.json", "format": "json", "size_bytes": 8192,
     "hash": "sha256:..."}
  ],
  "statistics": {
    "nodes": 320,
    "edges": 1250,
    "mean_confidence": 0.92,
    "validation_status": "PASS"
  }
}
```

Storing per-artifact SHA-256 hashes enables consumers to verify that exported bundles have not been modified after generation. When publishing datasets derived from COGANT exports, including the manifest alongside the data satisfies the reproducibility requirements outlined by [Peng, 2011]: any researcher with the same source tree, COGANT version, and configuration can regenerate the artifacts and compare hashes.

8.8 Data ethics and licensing

Exported graphs can contain identifiers and comments from source code. Redistribution of derived graphs must respect the licenses of input repositories and organizational data policies.

9 Scope and related work

9.1 Positioning

Machine learning for big code surveys cover naturalness, representation learning, and task taxonomy [Allamanis et al., 2018]. COGANT contributes **infrastructure**: a documented IR, pipeline, and export contract rather than a new benchmark or model architecture.

Graph neural networks unify message-passing frameworks for relational data [Wu et al., 2021, Scarselli et al., 2009]. Although COGANT's primary output is the Active Inference Institute's Generalized Notation Notation, its program graph can be materialised as optional tensor views compatible with these frameworks (PyG, DGL) so that graph-neural-network model papers can cite a specific tensor layout.

Code property graphs and related security-oriented graph constructions show how to merge syntactic and semantic edges for analysis [Yamaguchi et al., 2014]. COGANT's program graph is cognate but emphasizes **ML export**, confidence scoring, and multi-stage IR refinement.

9.2 Related tool categories

- **Compiler and LLVM IRs** — richer semantics, heavier toolchain; COGANT favors lightweight extraction for dataset building.
- **SCA and linters** — rule enforcement focus; COGANT focuses on graph generation for downstream learning.
- **Neural code models** (code2vec, CodeBERT, etc.) — often consume tokens or AST paths; COGANT supplies **explicit program graphs** that graph-neural-network-centric research can ingest directly.

9.3 Program analysis for ML

Several systems address the intersection of program analysis and machine learning that COGANT operates in:

code2seq and code2vec [Alon et al., 2019] represent programs as sets of AST paths between terminal nodes. These path-based representations are effective for method naming and code summarization but discard the full graph topology that graph-neural-network-based models exploit. COGANT preserves the complete program graph — including control-flow and data-flow edges — and can export it in tensor form, enabling downstream architectures that reason over richer relational structure.

Gated Graph Neural Networks (GGNN) [Li et al., 2016] introduced gated recurrent propagation for graph-structured program representations, demonstrating strong results on variable misuse detection and other code tasks. COGANT's export contract (PyG `Data` objects with typed `edge_index` and `edge_attr`) is directly compatible with GGNN-style models; the kind and role indices provide the discrete node and edge types that typed message-passing layers require.

CodeQL (Semmle/GitHub) provides a declarative query language over relational representations of code. While CodeQL excels at security analysis with hand-written queries, its outputs are query results rather than tensor-ready graph bundles. COGANT occupies the complementary niche: it

produces the graph data that learned models consume, and can ingest CodeQL query results as an additional evidence source feeding the confidence model.

CodeBERT [Feng et al., 2020] and related pre-trained models operate at the token level, learning representations from natural language and code jointly. These models are complementary to COGANT's graph-centric approach: CodeBERT embeddings could serve as optional node features in COGANT's Generalized Notation Notation (GNN) export (the export schema already reserves dimensions for text embeddings as documented in `../cogant/docs/GNN_EXPORT.md`).

9.3.1 Feature matrix: COGANT vs. related tools

The following matrix contrasts COGANT's capabilities with the related tools discussed in this section. Entries marked " " indicate first-class support; "partial" indicates limited or indirect support; "—" indicates the feature is out of scope for that tool.

Table 4. Feature comparison of program-to-model toolchains.

Feature	COGANT	code2vec	GGNN	CodeQL	CodeBERT
Full program graph (AST + CFG + DFG)		—	input-only		—
Typed node/edge taxonomy		—	partial		—
Confidence scoring per assertion		—	—	—	—
Provenance tracking		—	—	partial	—
State-space extraction		—	—	—	—
Temporal regime classification		—	—	—	—
Dynamic enrichment (coverage, traces)		—	—	partial	—
Generalized Notation Notation output		—	—	—	—
Tensor export (PyG, DGL, HDF5)		partial	input-only	—	—
Pluggable translation rules		—	—		—
Human review loop		—	—	partial	—
Multi-language front-ends	roadmap		—		

COGANT is distinct from the other toolchains in three ways: first, it explicitly models uncertainty through confidence tiers tied to evidence provenance; second, it produces a structured Active Inference notation as its primary output rather than an opaque tensor; and third, it composes static and dynamic evidence in a single pipeline rather than specializing to one.

9.4 Active inference and program behavior

The state-space IR in COGANT's pipeline (states, actions, transitions, observations) shares structural parallels with **active inference** formulations [Friston, 2010], where an agent maintains beliefs about hidden states and selects actions to minimize prediction error. In the program analysis context, the "agent" is the analysis pipeline itself: it observes code artifacts, maintains beliefs about program behavior (the state-space model), and refines those beliefs as new evidence (dynamic traces, coverage data) arrives.

This connection is analogical: the `ConfidenceModel` in `../cogant/py/cogant/translate/confidence.py` aggregates evidence and penalties in a way that suggests belief revision, but it is not a Bayesian

posterior. Future work could formalize a tighter link by casting rule application as variational inference, where a fixpoint would represent an approximate posterior over program semantics.

9.5 Boundaries

COGANT does not subsume formal verification, interactive theorem proving, or full interprocedural pointer analysis unless implemented as explicit future stages. The SPEC marks Rust acceleration and additional parsers as staged; the manuscript should be read together with that table for up-to-date scope.

9.6 Forward compatibility

Promoting COGANT into `../../../projects/` integrates manuscript PDF rendering with the template's validation gates. Cross-references in this folder use paths **relative to these Markdown files** (for example `../cogant/docs/`) so links stay stable when the tree moves.

10 Manuscript Syntax Reference (COGANT)

Formatting conventions for Markdown in this folder. For the full rendering contract, see the template exemplar projects/code_project/manuscript/SYNTAX.md.

10.1 Citations

Use Pandoc cite syntax; keys must exist in `references.bib`.

[@allamanis2018survey]

[@allamanis2018survey; @wu2020comprehensive]

10.2 Equations

Use LaTeX `equation` with `\label` / `\ref` as documented in the code_project SYNTAX, or pandoc-crossref attributes where the pipeline enables them.

10.3 Figures

If you add figures, place assets where the future project `output/` layout can resolve them (typically `../figures/` after promotion to `projects/cogant/`). Use explicit relative paths from the rendering contract described in `infrastructure/rendering/AGENTS.md`.

10.4 Section files

Numeric prefixes `00_`–`09_` are combined in stem-sorted order by `infrastructure/rendering/manuscript_discovery.py`. Files named `SYNTAX.md` sort in the **other** bucket (after main sections, before `99_*` if present).

10.5 Cross-references to the package

Prefer **relative** paths from this folder to the package tree, e.g. `../cogant/docs/ARCHITECTURE.md`, so links work in the editor and in Git without hard-coding the monorepo path.

10.6 See also

- `AGENTS.md` — editor protocol for this folder
- `README.md` — orientation and render notes

References

Active Inference Institute. Generalized notation notation (gnn): A structured notation for active inference state-space and process models. https://github.com/ActiveInferenceInstitute/GeneralizedNotationNotation, 2024. Open-source specification and reference implementation; not to be confused with graph neural networks.

Miltiadis Allamanis, Earl T. Barr, Premkumar Devanbu, and Charles Sutton. A survey of machine learning for big code and naturalness. *ACM Computing Surveys*, 51(3):1–37, 2018. doi: 10.1145/3212695.

Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. code2vec: Learning distributed representations of code. In *Proceedings of the ACM on Programming Languages (POPL)*, volume 3, pages 1–29, 2019. doi: 10.1145/3290353.

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, 2020. doi: 10.18653/v1/2020.findings-emnlp.139.

Karl Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11 (2):127–138, 2010. doi: 10.1038/nrn2787.

Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. Gated graph sequence neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016. URL https://arxiv.org/abs/1511.05493.

Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.

Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. In *IEEE International Joint Conference on Neural Networks*, pages 1–8, 2009. doi: 10.1109/IJCNN.2009.5178198.

Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021. doi: 10.1109/TNNLS.2020.2978386.

Fabian Yamaguchi, Nico Golde, Sascha Arzt, and Konrad Rieck. Modeling and discovering vulnerabilities with code property graphs. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy*, pages 590–604, 2014. doi: 10.1109/SP.2014.43.

Pengcheng Yin, Graham Neubig, Miltiadis Allamanis, Marc Brockschmidt, and Alexander L. Gaunt. Learning to represent edits. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019. URL https://openreview.net/forum?id=BJl6AjC5F7.